

Complete AI Coding Assistant Command Reference

The Full 10-Repo Stack: Optimization, Workflow, Context, Minimalism, Mapping, Knowledge & Design

This is the definitive reference covering all 10 essential AI agent frameworks: **ECC** • **Superpowers** • **GSD Core** • **Ponytail** • **Graphify** • **claude-obsidian** • **obsidian-skills** • **andrej-karpathy-skills** • **ui-ux-pro-max-skill** • **open-design**

Complete Commands, Hooks, Skills, and Agents Reference

ECC + Superpowers + GSD Core - All Commands Listed

1. ECC (affaan-m/ECC) - 231K+ Stars

Agent Harness Performance Optimization System

Installation Commands

```
/plugin marketplace add https://github.com/affaan-m/ECC
/plugin install ecc@ecc
npx ecc consult "mlops training model deployment" --target claude
npx ecc install --profile minima
npx ecc install --profile core
npx ecc install --profile full
./install.sh --profile full
.\install.ps1 --profile full
npx ecc-install --profile full
```

Slash Commands (Maintained)

Command	Purpose
<code>/ecc:plan</code>	Feature implementation planning
<code>/ecc:code-review</code>	Code quality and security review
<code>/ecc:build-fix</code>	Fix build and compile errors
<code>/ecc:refactor-clean</code>	Remove dead code and simplify
<code>/ecc:quality-gate</code>	Run verification gate checks
<code>/ecc:learn</code>	Extract patterns from session
<code>/ecc:checkpoint</code>	Save verification state
<code>/ecc:security-scan</code>	Run AgentShield security auditor
<code>/ecc:update-docs</code>	Update documentation automatically
<code>/ecc:update-codemaps</code>	Update codebase maps
<code>/ecc:test-coverage</code>	Analyze test coverage
<code>/ecc:go-review</code>	Go code review
<code>/ecc:go-test</code>	Go TDD workflow
<code>/ecc:go-build</code>	Fix Go build errors
<code>/ecc:python-review</code>	Python code review (PEP 8)
<code>/ecc:setup-pm</code>	Configure package manager
<code>/ecc:skill-create</code>	Generate skills from git history
<code>/ecc:instinct-status</code>	View learned instincts
<code>/ecc:instinct-import</code>	Import instincts from file
<code>/ecc:instinct-export</code>	Export your instincts
<code>/ecc:evolve</code>	Cluster instincts into skills
<code>/ecc:prune</code>	Delete expired pending instincts
<code>/ecc:pm2</code>	PM2 service lifecycle management

/ecc:multi-plan	Multi-agent task decomposition
/ecc:multi-execute	Multi-agent orchestrated workflows
/ecc:multi-backend	Backend multi-service orchestration
/ecc:multi-frontend	Frontend multi-service orchestration
/ecc:multi-workflow	General multi-service workflows
/ecc:sessions	Session history management
/ecc:harness-audit	Audit harness reliability
/ecc:loop-start	Start controlled agentic loop
/ecc:loop-status	Inspect active loop status
/ecc:model-route	Route tasks to models by complexity
/ecc:learn-eval	Extract and evaluate patterns
/ecc:promote	Promote project instincts globally
/ecc:projects	List projects and instinct stats

Legacy Commands (Still Available - Backward Compatible)

/plan	→ now /ecc:plan
/code-review	→ now /ecc:code-review
/build-fix	→ now /ecc:build-fix
/refactor-clean	→ now /ecc:refactor-clean
/quality-gate	→ now /ecc:quality-gate
/learn	→ now /ecc:learn
/checkpoint	→ now /ecc:checkpoint
/update-docs	→ now /ecc:update-docs
/test-coverage	→ now /ecc:test-coverage
/go-review	→ now /ecc:go-review
/go-test	→ now /ecc:go-test
/go-build	→ now /ecc:go-build
/python-review	→ now /ecc:python-review
/sessions	→ now /ecc:sessions
/skill-create	→ now /ecc:skill-create
/instinct-status	→ now /ecc:instinct-status
/instinct-import	→ now /ecc:instinct-import
/instinct-export	→ now /ecc:instinct-export
/evolve	→ now /ecc:evolve

```

/prune          → now /ecc:prune
/pm2            → now /ecc:pm2
/multi-plan     → now /ecc:multi-plan
/multi-execute → now /ecc:multi-execute
/multi-backend  → now /ecc:multi-backend
/multi-frontend → now /ecc:multi-frontend
/multi-workflow → now /ecc:multi-workflow
/setup-pm       → now /ecc:setup-pm

```

CLI Commands

```

npx ecc consult "topic" --target claude
npx ecc install --profile minima
npx ecc install --profile full --target claude
npx ecc status --markdown --write status.md
npx ecc status --exit-code
ecc work-items upsert ...
ecc work-items sync-github --repo owner/repo
node scripts/ecc.js list-installed
node scripts/ecc.js doctor
node scripts/ecc.js repair
node scripts/ecc.js uninstall --dry-run
node scripts/uninstall.js --dry-run
node scripts/uninstall.js
node scripts/setup-package-manager.js --global pnpm
node scripts/setup-package-manager.js --project bun
node scripts/setup-package-manager.js --detect
npx ecc-agentshield scan
npx ecc-agentshield scan --fix
npx ecc-agentshield scan --opus --stream
npx ecc-agentshield init
npm run dashboard
python3 ./ecc_dashboard.py

```

Environment Variables for Configuration

```

export ECC_HOOK_PROFILE=standard # Hook strictness: minimal|standard|
export ECC_DISABLED_HOOKS="hook1,hook2" # Comma-separated hook IDs to disable
export ECC_SESSION_START_MAX_CHARS=8000 # SessionStart context limit (default)
export ECC_SESSION_START_CONTEXT=off # Disable SessionStart context entirely
export ECC_SESSION_RETENTION_DAYS=30 # Session retention window (0/off/full)
export ECC_MAX_INJECTED_INSTINCTS=6 # Max instincts injected at session start
export ECC_INSTINCT_CONFIDENCE_THRESHOLD=0.7 # Min confidence for instinct injection
export ECC_CONTEXT_MONITOR_COST_WARNINGS=off # Suppress API-rate cost estimates
export ECC_AGENT_DATA_HOME="$HOME/.cursor/ecc" # Multi-harness memory isolation

```

```
export ECC_DISABLED_MCPS="chrome-devtools"    # MCPs to skip during install/sync
export CLAUDE_PACKAGE_MANAGER=pnpm           # Preferred package manager
```

Hooks (Trigger-Based Automations)

PreToolUse	→ Before tool execution
PostToolUse	→ After tool execution
Stop	→ On session stop
SessionStart	→ On session start
SessionEnd	→ On session end
Pre-compact	→ Pre-compaction state saving
Suggest-compact	→ Strategic compaction suggestions
Evaluate-session	→ Extract patterns from sessions

Agents (67 Total - Specialized Subagents)

```
planner.md
architect.md
tdd-guide.md
code-reviewer.md
security-reviewer.md
build-error-resolver.md
e2e-runner.md
refactor-cleaner.md
doc-updater.md
docs-lookup.md
chief-of-staff.md
loop-operator.md
harness-optimizer.md
cpp-reviewer.md, cpp-build-resolver.md
fsharp-reviewer.md
go-reviewer.md, go-build-resolver.md
python-reviewer.md
database-reviewer.md
typescript-reviewer.md
java-reviewer.md, java-build-resolver.md
kotlin-reviewer.md, kotlin-build-resolver.md
harmonyos-app-resolver.md
rust-reviewer.md, rust-build-resolver.md
pytorch-build-resolver.md
mle-reviewer.md
... and 40+ more domain-specific agents
```

Skills (278 Total Categories)

coding-standards/
backend-patterns/
frontend-patterns/
frontend-slides/
article-writing/
content-engine/
market-research/
investor-materials/
investor-outreach/
tdd-workflow/
security-review/
eval-harness/
verification-loop/
continuous-learning/
continuous-learning-v2/
iterative-retrieval/
strategic-compact/
videodb/
golang-patterns/, golang-testing/
cpp-coding-standards/, cpp-testing/
django-patterns/, django-security/, django-tdd/, django-verification/
laravel-patterns/, laravel-security/, laravel-tdd/, laravel-verification/
python-patterns/, python-testing/
springboot-patterns/, springboot-security/, springboot-tdd/, springboot-verification/
quarkus-patterns/, quarkus-security/, quarkus-tdd/, quarkus-verification/
configure-ecc/
security-scan/
database-migrations/
api-design/
deployment-patterns/
docker-patterns/
e2e-testing/
... and 200+ more skills

Rules (34 Total - Always-Follow Guidelines)

rules/common/
├ coding-style.md
├ git-workflow.md
├ testing.md
├ performance.md
├ patterns.md
├ hooks.md
├ agents.md
└ security.md

```
rules/typescript/  
rules/python/  
rules/golang/  
rules/swift/  
rules/php/  
rules/arkts/
```

2. Superpowers (obra/superpowers) - 6.3K Stars

Agentic Skills Framework & Software Development Methodology

Installation Commands

```
/plugin marketplace add obra/superpowers-marketplace  
/plugin install superpowers@superpowers-marketplace  
/plugins install https://github.com/obra/superpowers  
  
# For Codex  
copilot plugin marketplace add obra/superpowers-marketplace  
copilot plugin install superpowers@superpowers-marketplace  
  
# For Kimi Code  
/plugins install https://github.com/obra/superpowers  
# Then: /plugins → Marketplace > Superpowers  
  
# For OpenCode  
pi install git:github.com/obra/superpowers  
  
# Manual/Development  
cd superpowers && npm install
```

Slash Commands (Built-in Workflows)

Command	Purpose
/brainstorm	Explore ideas without implementation
/write-plan	Create detailed implementation plan
/execute-plan	Execute planned work with parallel tasks

<code>/think-pair-share</code>	Collaborative decision making
<code>/preserving-productive-tensions</code>	Keep multiple valid approaches

Marketplace Commands

```

/plugin marketplace add obra/superpowers-marketplace
/plugin marketplace add obra/superpowers-developing-for-claude-code
/plugin marketplace add obra/superpowers-lab

```

Skills (20+ Battle-Tested)

```

brainstorm-skill      → Problem-space exploration
write-plan-skill     → Implementation phasing
execution-skill       → Execution methodology
code-review-skill     → Code quality review
testing-skill         → Test-first development
debugging-skill       → Systematic debugging
collaboration-skill   → Team coordination
meta-skill            → Process reflection
search-first-skill    → Research before coding
tdd-skill             → Test-driven development
security-skill        → Security verification
documentation-skill   → Doc generation
performance-skill     → Performance analysis
accessibility-skill   → Accessibility review
refactoring-skill     → Code refactoring
decision-logging-skill → Decision capture
estimation-skill      → Effort estimation
... and 2+ more

```

Available Plugins/Extensions

```

elements-of-style     → Writing clarity and conciseness (Strunk Jr. rules)
claude-code-development → Claude Code plugin development
private-journal-mcp    → Semantic search journaling
... and community extensions

```

Lab (Experimental Features)

```

/plugin install https://github.com/obra/superpowers-lab

```


Features:

- Finding duplicate functions (semantic deduplication)
- On-demand MCP server execution
- Experimental skills under active development

Community Skills Repository

```
# Skills auto-cloned to ~/.config/superpowers/skills/  
# Users can fork and submit PRs with improvements  
# Skills auto-update on session start
```

3. GSD Core (open-gsd/gsd-core) - 6.7K Stars

Context-Engineering & Spec-Driven Development Framework

Installation Commands

```
npx @opengsd/gsd-core@latest --install  
npx @opengsd/gsd-core@latest --claude --global  
npx @opengsd/gsd-core@latest --opencode --global  
npx @opengsd/gsd-core@latest --codex --global  
npx @opengsd/gsd-core@latest --cursor --global  
npx @opengsd/gsd-core@latest --gemini --global  
npx @opengsd/gsd-core@latest --copilot --global
```

Main Commands

Command	Purpose
/gsd-new-project	Initialize greenfield project
/gsd-onboard	Onboard existing codebase
/gsd-sync	Sync GSD with current state
/gsd-status	Check project status

Phase Loop Commands (5-Step Methodology)

Discuss → Capture implementation decisions
Plan → Research, decompose, verify
Execute → Run plans in parallel waves
Test → Verify and validate
Ship → Deploy and monitor

Workflow Commands

```
/gsd-plan-feature "Add new API endpoint"  
/gsd-plan-bugfix "Fix authentication issue"  
/gsd-estimate "effort in hours"  
/gsd-review "code changes"  
/gsd-ready-to-ship "final checks"
```

Generated Project Artifacts

```
.planning/  
├─ codebase/  
│   ├── STACK.md          (Tech stack detected)  
│   ├── CONVENTIONS.md    (Code patterns observed)  
│   └─ CONCERNS.md        (Technical debt & fragile areas)  
├─ phases/  
│   ├── 01-01-PLAN.md     (Current phase plan)  
│   ├── 01-02-EXECUTE.md  (Execution checklist)  
│   └─ ... (history of all phases)  
└─ decisions/  
    ├── ADR-001.md        (Architecture decisions)  
    └─ ... (decision log)
```

Build/Test Commands

```
pnpm install --frozen-lockfile  
pnpm run build  
pnpm run test  
pnpm run verify:fast      # CI fast-gates locally  
pnpm run verify:pr        # Fast loop: build + typecheck + unit tests  
pnpm run verify:merge     # Full CI blocking parity before PR review
```

Subagent Orchestration

```
Fresh-context subagents for:
├─ Research phase
├─ Planning phase
├─ Execution waves (parallel)
├─ Verification phase
└─ Shipping phase
```

Environment Configuration

```
# Runtime selection during install
--claude           # Claude Code
--opencode         # OpenCode/Antigravity
--codex            # OpenAI Codex CLI/App
--cursor           # Cursor IDE
--gemini           # Google Gemini CLI
--copilot          # GitHub Copilot (VS Code)
--global           # System-wide installation
--local            # Project-specific installation
```

Agents Available in GSD

```
gsd-planner      → Planning and decomposition
gsd-executor      → Execution and implementation
gsd-reviewer      → Code and design review
gsd-researcher    → Research and context gathering
gsd-shipper       → Deployment and monitoring
```

CLI Utilities

```
npx @opengsd/gsd-core --version
npx @opengsd/gsd-core --help
npx @opengsd/gsd-core doctor      # Check GSD health
npx @opengsd/gsd-core repair     # Auto-repair issues
npx @opengsd/gsd-core reset      # Reset to defaults
npx @opengsd/gsd-core export-state # Export project state
npx @opengsd/gsd-core import-state # Import project state
```

Quick Comparison Table

Feature	ECC	Superpowers	GSD Core
Total Commands	94+	5+ slash	5+ slash
Total Agents	67	~10	5
Total Skills	278+	20+	Integrated
Hook Types	8	Custom	N/A
Installation Methods	3+	2	7
Cross-Platform	Yes (7 harnesses)	Yes (5 harnesses)	Yes (6 harnesses)
Primary Focus	Agent optimization	Workflow methodology	Context management
Best For	Large codebases	Team collaboration	Long-running tasks

How to Use These Commands

ECC Workflow

1. `/ecc:plan "feature description"`
2. `tdd-workflow skill`
3. `/ecc:code-review`
4. `/ecc:security-scan`
5. `/ecc:quality-gate`
6. `Deploy`

Superpowers Workflow

1. `/brainstorm "problem space"`
2. `/write-plan "implementation"`
3. `/execute-plan "build it"`
4. `Team review`
5. `Ship`

GSD Core Workflow

1. `/gsd-new-project` or `/gsd-onboard`

2. `/gsd-plan-feature`
 3. Parallel execution waves
 4. Verification phase
 5. `/gsd-ready-to-ship`
-

Installation Checklist

For ECC

- `/plugin marketplace add https://github.com/affaan-m/ECC`
- `/plugin install ecc@ecc`
- Copy rules to `~/.claude/rules/ecc/`
- Set environment variables
- Verify with `/plugin list ecc@ecc`

For Superpowers

- `/plugin marketplace add obra/superpowers-marketplace`
- `/plugin install superpowers@superpowers-marketplace`
- Run `/brainstorm` or `/write-plan` to test
- Explore available skills

For GSD Core

- `npx @opengsd/gsd-core@latest --install`
 - Choose your runtime (Claude Code, OpenCode, etc.)
 - Run `/gsd-new-project` or `/gsd-onboard`
 - Follow the guided workflow
-

All commands tested and verified as of July 2026. Bookmark this reference for daily use!

4. Ponytail (DietrichGebert/ponytail) - 85K+ Stars

AI Agent “Lazy Senior Dev” Optimization & Minimal Code Generation

Description: Makes your AI agent think like the laziest senior developer in the room using YAGNI principle (You Ain't Gonna Need It). Forces minimal solutions, standard library first, no unnecessary abstractions. Philosophy: the best code is the code you never wrote.

Installation Commands

```
# Claude Code
/plugin marketplace add https://github.com/DietrichGebert/ponytail
/plugin install ponytail@ponytail

# OpenCode (reads AGENTS.md automatically)
/hooks review and trust lifecycle hooks
# Then run new thread

# Codex
codex plugin marketplace add DietrichGebert/ponytail
codex plugin add ponytail@ponytail
codex plugin install ponytail@ponytail

# Copilot (VS Code)
copilot plugin marketplace add DietrichGebert/ponytail
copilot plugin install ponytail@ponytail

# Copilot CLI
/plugin marketplace add DietrichGebert/ponytail
/plugin install ponytail@ponytail

# Swival Package Manager
swival skills add --global https://github.com/DietrichGebert/ponytail
swival skills add ponytail
swival skills add --global ponytail

# Manual Setup
copy AGENTS.md to project root
run codewhale from checkout
```

Slash Commands

Command	Purpose
<code>/ponytail</code>	Activate lazy mode (lite/full/ultra)
<code>/ponytail:ponytail lite</code>	Lite level laziness (minimal)
<code>/ponytail:ponytail full</code>	Full level laziness (default)
<code>/ponytail:ponytail ultra</code>	Ultra level laziness (most minimal)
<code>/ponytail:ponytail off</code>	Disable lazy mode
<code>/ponytail-review</code>	Review code with lazy principles
<code>/ponytail-audit</code>	Audit codebase for over-engineering
<code>/ponytail-debt</code>	Track technical debt & shortcuts
<code>/ponytail-gain</code>	Measure gains from lazy approach
<code>/ponytail-help</code>	Show ponytail help & quick reference

Activation Triggers (Auto-Activate)

"ponytail"
 "be lazy"
 "lazy mode"
 "simplest solution"
 "minimal solution"
 "yagni"
 "do less"
 "shortest path"
 "over-engineered"
 "bloated"
 "boilerplate"
 "unnecessary dependencies"

Core Skills (12 Total)

<code>ponytail-mode-selector</code>	→ Choose lite/full/ultra
<code>ponytail-yagni</code>	→ You Ain't Gonna Need It principle
<code>ponytail-stdlib-first</code>	→ Use standard library before custom
<code>ponytail-minimal-solution</code>	→ Shortest, simplest, most minimal code
<code>ponytail-debt-ledger</code>	→ Track shortcuts and deferrals
<code>ponytail-dependency-review</code>	→ Question every dependency

ponytail-lazy-refactor	→ Refactor minimally
ponytail-lazy-architecture	→ Design simply
ponytail-lazy-review	→ Review for over-engineering
ponytail-quick-reference	→ Commands and modes cheat sheet
ponytail-intensity-levels	→ lite (minimal) → full (default) → ultra
ponytail-senior-dev-mode	→ Channel 30-year-veteran developer

Intensity Levels

lite	→ Minimal suggestions, leave most to you
full	→ Default balance of suggestions and automation
ultra	→ Maximum lazy solution forcing
off	→ Completely disabled

Rules/Guidelines Injected Per Turn

YAGNI (You Ain't Gonna Need It)

- └ Question whether task needs to exist at all

Standard Library First

- └ Use stdlib before custom code
- └ Use native platform features before dependencies

Minimal/Shortest Path

- └ One line before fifty
- └ Remove boilerplate relentlessly
- └ No unrequested abstractions

Simplicity

- └ KISS (Keep It Simple, Stupid)
- └ Linear code over clever code
- └ Explicit over implicit (but minimal)

Use Cases (Coding Only)

- ✓ Writing new code
- ✓ Refactoring existing code
- ✓ Fixing bugs
- ✓ Code review
- ✓ Library/dependency selection
- ✓ Architecture design
- ✗ Non-coding requests (knowledge, prose, translation, summaries, recipes)

Project Configuration

```
AGENTS.md (project root)
└─ Auto-loaded by OpenCode
└─ Defines rules for this project
└─ Zero setup required if copied from repo
```

Benchmarks & Comparison

```
Real model output (Claude Haiku 4.5):
- Without Ponytail: Verbose, over-engineered solutions
- With Ponytail: Minimal, pragmatic solutions

Generate benchmarks yourself:
npx promptfoo@latest eval -c benchmarks/promptfooconfig.yaml
```

Environment Variables

```
export PONYTAIL_MODE=full           # Default mode
export PONYTAIL_INTENSITY=ultra     # Or: lite, full, ultra
export PONYTAIL_LEVEL=standard      # Hook strictness
export PONYTAIL_DISABLED=false
```

5. Graphify (Graphify-Labs/graphify) - 90K+ Stars

AI-Powered Code Knowledge Graph & Dependency Visualization

Description: Turn any folder of code, SQL schemas, R scripts, shell scripts, docs, papers, images, or videos into a queryable knowledge graph. Maps entire projects into persistent knowledge graphs showing how code actually fits together. Supports Claude Code, Codex, OpenCode, Cursor, Gemini CLI, and more.

Installation Commands

```
# Recommended: UV/Pipx
uv tool install graphify && graphify install
pipx install graphify && graphify install

# Then register skill
```

```
/graphify .

# Project-Scoped Installation
graphify install --project
graphify install --project --platform codex

# Platform-Specific Installs
graphify claude install
graphify claude install --project
graphify codex install
graphify codex install --project
graphify opencode install
graphify cursor install
graphify gemini install
graphify copilot install

# PowerShell Note
# Use: graphify . (not /graphify . - slash is path separator)
```

Main Slash Commands

Command	Purpose
/graphify	Map entire project into knowledge graph
/graphify query <query>	Query the knowledge graph
/graphify community-detect	Find community clusters/subsystems
/graphify verify	Verify a change is safe
/graphify diff	Show what changed in the graph
/graphify export	Export graph in various formats
/graphify label	Auto-name communities
/graphify help	Show graphify help

CLI Commands

```
# Core Operations
graphify install
graphify install --project
graphify run

# Register skill with assistant
# Project-scoped install
# Run extraction on current directory
```

graphify run path/to/codebase	# Extract specific path
graphify query "semantic search"	# Query the graph
graphify community-detect	# Cluster subsystems
graphify label	# Auto-name communities
graphify label --no-label	# Keep default naming (Community N)
# Export & Analysis	
graphify export json	# Export as JSON
graphify export graphml	# Export as GraphML
graphify export csv	# Export as CSV
graphify export html	# Generate interactive HTML
graphify export markdown	# Export as Markdown report
graphify export git	# Export git history
# Verification (Enterprise)	
graphify verify path/to/change	# Formal verification of changes
graphify diff	# Show graph differences
graphify blame	# Attribution analysis
# Configuration	
graphify config show	# Show current config
graphify config set <key> <value>	# Set configuration
graphify doctor	# Health check
graphify repair	# Auto-repair issues
# Advanced	
graphify extract	# Manual extraction
graphify cache clear	# Clear extraction cache
graphify debug	# Debug mode

Supported Input Types

Code

- └ Python
- └ TypeScript/JavaScript
- └ Java
- └ C++/C
- └ Go
- └ Rust
- └ Ruby
- └ PHP
- └ Swift
- └ Kotlin
- └ ... and more

Structured Data

- └ SQL schemas
- └ YAML configs
- └ JSON configs
- └ GraphQL schemas

Documents

- └ Markdown docs
- └ Plain text
- └ Papers/PDFs
- └ API documentation

Media

- └ Images (vision-capable extraction)
- └ Videos (transcript extraction)
- └ Audio (transcript extraction)

Version Control

- └ Git history
- └ Commit messages
- └ Branch structures

Generated Output Artifacts

graphify-out/

- └ graph.html
 - └ Interactive web visualization
 - └ Click nodes, filter, search
 - └ Zoom and pan
- └ GRAPH_REPORT.md
 - └ Highlights and key insights
 - └ Community breakdown
 - └ Top nodes (god nodes)
- └ graph.json
 - └ Machine-readable format
- └ graph.graphml
 - └ GraphML format (Neo4j compatible)
- └ WIKI.md
 - └ Auto-generated wiki
- └ query_results.json
 - └ Search results

Graph Analysis Features

Extraction Pipeline

- └ Parse source code/docs
- └ Extract symbols, references, patterns
- └ Build edges (calls, imports, dependencies)
- └ Store in knowledge graph

Community Detection (Leiden Algorithm)

- └ Cluster related components
- └ Auto-label communities
- └ Identify subsystems
- └ Show interdependencies

Confidence Scoring

- └ Tag edges as EXTRACTED or INFERRED
- └ Show uncertainty levels
- └ Explain reasoning
- └ Let user decide trust

God Nodes Detection

- └ Find central hubs
- └ Identify single points of failure
- └ Surface coupling risks
- └ Guide refactoring

Edge Attribution

EXTRACTED → Explicitly in source code

INFERRED → Resolved by graphify analysis

Example:

- EXTRACTED: Direct function call
- INFERRED: Polymorphic dispatch that graphify resolved

Platform-Specific Install Paths

Claude Code

- └ ~/.claude/skills/graphify/SKILL.md
- └ ~/.claude/skills/graphify/references/

Codex

- └ ~/.config/codex/skills/graphify/SKILL.md

OpenCode
└─ ~/.opencode/skills/graphify/SKILL.md

Cursor
└─ ~/.cursor/skills/graphify/SKILL.md

Gemini CLI
└─ ~/.gemini/skills/graphify/SKILL.md

Configuration Options

```
export GRAPHIFY_BACKEND=openai      # LLM backend for labeling
export GRAPHIFY_MODEL=gpt-4o        # Model for community naming
export GRAPHIFY_MAX_CONTEXT=200000  # Max context for analysis
export GRAPHIFY_CACHE_DIR=.graphify # Cache location
export GRAPHIFY_LANGUAGES=python,ts # Restrict to languages
export GRAPHIFY_DEPTH=3              # Traversal depth
export GRAPHIFY_MIN_CONFIDENCE=0.7   # Min confidence threshold
export GRAPHIFY_DOCKER_BACKEND=true  # Use Docker for extraction
export GRAPHIFY_SQLITE_BACKEND=true  # Use SQLite persistence
```

Advanced Features

Optional Integrations

- └─ Docker MCP Toolkit (containerized extraction)
- └─ SQLite backend (persistent graphs)
- └─ Custom language extractors
- └─ API integrations
- └─ Git hooks automation

Verification Layer (Enterprise)

- └─ Formal verification of code changes
- └─ Input/output proof generation
- └─ Regression detection
- └─ Safety guarantees
- └─ Runnable test cases

Persistent Memory

- └─ Session memory layer
- └─ Incremental updates
- └─ Historical graph snapshots
- └─ Change tracking

Associated Tools

Penpax

- └ Always-on memory layer built on Graphify
- └ Extends to: meetings, browser history, emails, files, code
- └ Auto-updating working graph of your entire life

The Memory Layer (Book)

- └ Complete guide to building persistent memory for AI agents
- └ Architecture of knowledge graphs
- └ Graphify internals deep dive

Performance Benchmarks

Extraction Speed

- └ Average: 1000 LOC per second
- └ Python: 800 LOC/s
- └ TypeScript: 1200 LOC/s

Graph Size

- └ 50K LOC → ~5000 nodes/edges
- └ 500K LOC → ~50K nodes/edges
- └ 5M LOC → ~500K nodes/edges

Query Speed

- └ Simple query: <100ms
- └ Complex traversal: <500ms
- └ Full graph analysis: 1-5s

Verification Examples (Enterprise)

Before: Code change + guess it's safe

After: Graphify proves: "This change is safe" OR
"Breaking input: {example} fails with {error}"

Verified Properties

- └ Type safety
 - └ Control flow
 - └ Data dependencies
 - └ Side effects
 - └ Invariants
-
-
-

1. claude-obsidian (AgriciDaniel/claude-obsidian) - 9.4K+ Stars

Self-Organizing AI Second Brain for Obsidian + Claude Code

Description: A running AI notetaker that builds and maintains a persistent, compounding wiki vault. Drop any source and Claude reads, links, and files it into a connected knowledge graph of plain Markdown. Based on Andrej Karpathy's LLM Wiki pattern. 15 Claude Code skills with multi-agent support.

Installation Commands

```
# Option 1: Clone as vault (recommended, 2-min setup)
git clone https://github.com/AgriciDaniel/claude-obsidian
cd claude-obsidian
bash bin/setup-vault.sh
# Then open in Obsidian, open Claude Code, type /wiki

# Option 2: Install as Claude Code plugin
claude plugin marketplace add AgriciDaniel/claude-obsidian
claude plugin install claude-obsidian@agricidaniel-claude-obsidian
claude plugin list # verify install

# Option 3: Add to existing vault
# Copy WIKI.md into vault root, then paste setup prompt into Claude
```

Core Commands

Command	Purpose
/wiki	Setup check, scaffold, or continue where you left off
ingest [file]	Read source, create 8-15 wiki pages, update index/log
ingest all of these	Batch process multiple sources, then cross-reference
what do you know about X?	Read index, drill into pages, synthesize cited answer
/save	File current conversation as a wiki note
/save [name]	Save with specific title (skips naming question)

<code>/autoresearch [topic]</code>	Autonomous research loop: search, fetch, synthesize, file
<code>/canvas</code>	Open/create visual canvas, list zones and nodes
<code>/think [problem]</code>	Apply 10-principle thinking loop to a problem
<code>lint the wiki</code>	Health check: orphans, dead links, gaps, suggestions
<code>update hot cache</code>	Refresh hot.md with latest context summary

Canvas Sub-Commands

```

/canvas                # open or create the canvas
/canvas add image <path> # add image with auto-layout
/canvas add pdf <path>  # render PDF as preview node
/canvas add note <wiki-page> # pin wiki page as linked card
/canvas zone <name>     # add labeled zone
/canvas from banana     # capture recent AI-generated images

```

15 Claude Code Skills

```

wiki/                # orchestrator + references
wiki-ingest/         # source ingestion
wiki-query/          # answer questions from vault
wiki-lint/           # vault health check
wiki-cli/            # Obsidian CLI transport (v1.7+)
wiki-retrieve/       # hybrid retrieval (v1.7+, opt-in)
wiki-mode/           # methodology modes router (v1.8+)
wiki-fold/           # log rollup (DragonScale opt-in)
save/                # /save: file conversations to wiki
autoresearch/        # autonomous research loop
canvas/              # visual layer (images, PDFs, notes)
defuddle/            # web extraction wrapper
obsidian-bases/      # Bases schema reference
obsidian-markdown/   # OFM syntax reference
think/              # 10-principle thinking framework

```

3 Specialized Agents

```

verifier.md          → Pre-commit audit agent (six-cut review)
wiki-ingest.md        → Parallel batch ingestion agent
wiki-lint.md          → Health check agent

```

Methodology Modes (v1.8+)

Generic (default)	→ No opinion, v1.7 behavior
LYT	→ Linking Your Thinking (notes link, MOCs navigate)
PARA	→ Projects/Areas/Resources/Archives (Tiago Forte)
Zettelkasten	→ Atomic notes, unique IDs, no folders

MCP Setup Commands

```
# Option A: REST API based
claude mcp add-json obsidian-vault '{...OBSIDIAN_API_KEY...}' --scope user

# Option B: Filesystem based (no plugin needed)
claude mcp add-json obsidian-vault '{
  "type": "stdio",
  "command": "npx",
  "args": ["-y", "@bitbonsai/mcpvault@latest", "/path/to/vault"]
}' --scope user
```

Uninstall

```
claude plugin uninstall claude-obsidian@agricidaniel-claude-obsidian
claude plugin marketplace remove AgriciDaniel/claude-obsidian
make clean-test-state # clear runtime state without uninstalling
```

Related Companion Tools

claude-canvas	→ Visual canvas orchestration (12 templates, 6 layouts)
claude-ads	→ Multi-platform paid ads audit (250+ checks)
claude-seo	→ Technical SEO + GEO audit suite

2. obsidian-skills (kepano/obsidian-skills) - 42K+ Stars

Agent Skills for Obsidian CLI & Open Formats

Description: Agent Skills for use with Obsidian, built by Obsidian's creator (Steph Ango / kepano). Teaches agents to use the Obsidian CLI and open formats including Markdown,

Bases, and JSON Canvas. Follows the Agent Skills specification for compatibility with Claude Code, Codex, and OpenCode.

Installation Commands

```
# Claude Code Marketplace
/plugin marketplace add kepano/obsidian-skills
/plugin install obsidian@obsidian-skills

# npx skills (SSH)
npx skills add git@github.com:kepano/obsidian-skills.git

# npx skills (HTTPS)
npx skills add https://github.com/kepano/obsidian-skills

# Manual - Claude Code
# Copy repo contents into /.claude folder at vault root

# Manual - Codex
# Copy skills/ directory into ~/.codex/skills

# Manual - OpenCode
git clone https://github.com/kepano/obsidian-skills.git ~/.opencode/skills/obsidian
# Note: clone full repo, not just inner skills/ folder
```

Core Focus Areas (Skills Cover)

```
Obsidian CLI operations
Markdown (Obsidian Flavored Markdown / OFM) syntax
Bases (native database views, v1.9.10+)
JSON Canvas 1.0 spec
Vault navigation and structure
Note linking conventions
```

Compatibility

```
Claude Code    → Native skill support
Codex          → Agent Skills spec compatible
OpenCode       → Auto-discovers SKILL.md files
```

3. andrej-karpathy-skills (multica-ai/andrej-karpathy-

skills) - 192K+ Stars

Karpathy-Derived Claude Code Behavior Guidelines

Description: A single CLAUDE.md file (now a full Claude Code plugin) that improves Claude Code behavior, derived from Andrej Karpathy's documented observations on LLM coding pitfalls. Addresses silent assumptions, over-engineering, and lack of goal-driven execution. Originally forrestchang/andrej-karpathy-skills (redirects to multica-ai now).

Installation Commands

```
# Direct CLAUDE.md append (original method)
echo "" >> CLAUDE.md
curl https://raw.githubusercontent.com/forrestchang/andrej-karpathy-skills/main/CLAUDE.md >> CLAUDE.md

# Claude Code Plugin (current recommended method)
/plugin marketplace add multica-ai/andrej-karpathy-skills
/plugin install andrej-karpathy-skills@multica-ai

# Cursor (via committed project rule)
# Rule at .cursor/rules/karpathy-guidelines.mdc auto-applies
# See CURSOR.md for setup in other projects
```

The 4 Core Principles (Karpathy's Observations)

Principle	What It Fixes
Goal-Driven Execution	Transform imperative instructions into declarative success criteria with verification loops (not step-by-step micromanagement)
No Silent Assumptions	Models wrongly assume things on your behalf without checking; forces clarification-seeking
Anti-Overcomplication	Fixes tendency to bloat abstractions, build 1000-line solutions when 100 lines would do
Comment/Code Preservation	Stops models from changing/removing comments and code they don't fully understand

Skill Structure

```
skills/karpathy-guidelines/SKILL.md → Structured skill definition
```

.claude-plugin/plugin.json	→ Plugin manifest
.claude-plugin/marketplace.json	→ Marketplace distribution
.cursor/rules/karpathy-guidelines.mdc	→ Cursor project rule
CURSORS.md	→ Cursor setup guide
EXAMPLES.md	→ Usage examples
README.zh.md	→ Chinese translation

Usage Pattern

```
# Merge with project-specific instructions in CLAUDE.md:

## Project-Specific Guidelines
- Use TypeScript strict mode
- All API endpoints must have tests
- Follow existing error handling patterns in src/utils/errors.ts

# These guidelines bias the agent toward Karpathy's principles
# while respecting your project's specific constraints
```

Key Quote (Philosophy)

"LLMs are exceptionally good at looping until they meet specific goals... Don't tell it what to do, give it success criteria and watch it go."

4. ui-ux-pro-max-skill (nextlevelbuilder/ui-ux-pro-max-skill) - 107K+ Stars

AI Design Intelligence for Professional UI/UX

Description: AI-powered design intelligence toolkit providing searchable databases of UI styles, color palettes, font pairings, chart types, and UX guidelines. Works as a skill for Claude Code, Cursor, Windsurf, Copilot, and 18+ platforms.

Installation Commands

```
# Claude Code Marketplace (2-command install)
/plugin marketplace add nextlevelbuilder/ui-ux-pro-max-skill
/plugin install ui-ux-pro-max@ui-ux-pro-max-skill

# NPM CLI (Universal - works for all 18 platforms)
```

```
npm install -g ui-ux-pro-max-cli
uipro init --ai claude

# Or without global install
npx ui-ux-pro-max-cli init --ai {{platform}}

# Platform-specific examples
uipro init --ai claude --global      # Install to ~/.claude/skills/
uipro init --ai cursor --global      # Install to ~/.cursor/skills/
uipro init --offline                  # Compatibility flag, bundled templates
```

CLI Commands

Command	Purpose
uipro init --ai <platform>	Install skill for specified platform
uipro versions	List available versions
uipro update	Refresh skill files from installed CLI package
uipro uninstall	Remove skill (auto-detect platform)
uipro uninstall --ai claude	Remove from specific platform
uipro uninstall --global	Remove from global install

Search Command (Core Functionality)

```
python3 src/ui-ux-pro-max/scripts/search.py "<query>" --domain <domain> [-n <max_results>]
# On Windows, use "python" instead of "python3"
```

Design Database Coverage (v2.11.0)

- 84 UI styles
- 192 color palettes
- 74 font pairings
- 192 product types (with reasoning rules)
- 99 UX guidelines
- 104 icon entries
- 16 GSAP motion presets
- 25 chart types
- 22 technology stacks supported

Supported Tech Stacks (22 Total)

React, Next.js, Vue, Nuxt, Svelte, Astro
SwiftUI, React Native, Flutter
Tailwind, shadcn/ui
Jetpack Compose (Android)
Angular, Laravel, JavaFX
WPF, WinUI, Avalonia, Uno Platform, UWP
Three.js, HTML/CSS

Supported AI Platforms (18 Total)

claude, cursor, windsurf, copilot, kiro, roocode
kilocode, codex, qoder, gemini, trae, opencode
continue, codebuddy, droid, warp, augment
antigravity, openclaw

7 Skill Categories

banner-design/	→ Banner and header graphics
brand/	→ Brand identity generation, logos, CIP
design-system/	→ Design token architecture
design/	→ Core UI/UX design intelligence
slides/	→ Presentation slide design
ui-styling/	→ Component styling patterns
ui-ux-pro-max/	→ Main searchable database skill

Premium Tier Features (uupm.cc)

Extended Brand Design Skills	→ Logo, CIP, Banners, Iconography
Advanced Asset Creation	→ AI image generation for real assets
Enterprise Architecture	→ Scalable design tokens for teams
Priority Support	→ Dedicated technical assistance

Development Commands

```
cd cli
npm run sync:assets    # mirrors src/ -> cli/assets/ and .claude/skills/
npm run check:assets  # verify sync, no install required
```

5. open-design (nexu-io/open-design) - 79K+ Stars

The Open-Source Claude Design Alternative

Description: Local-first, open-source alternative to Anthropic's Claude Design. Turns your existing coding agent (Claude Code, Codex, Cursor, Gemini, OpenCode, Qwen, and 20+ CLIs) into a design engine that generates prototypes, landing pages, dashboards, slides, images, and video with real file exports (HTML/PDF/PPTX/MP4).

Installation Commands

```
# Full local development setup
git clone https://github.com/nexu-io/open-design.git
cd open-design
corepack enable && pnpm install
pnpm tools-dev run web

# Requirements: Node ~24, pnpm 10.33.x
# WSL2 users: see docs/wsl-setup.md
# Native Windows: see docs/windows-troubleshooting.md

# Quick contribution flow (non-coders)
npx skills add nexu-io/od-contribute
```

CLI Commands (`od` command - runs without UI)

Command	Purpose
<code>od plugin list</code>	List installed plugins (<code>-task-kind</code> / <code>-mode</code> / <code>-tag</code> filters)
<code>od plugin search "<query>"</code>	Search plugins by keyword
<code>od plugin info <plugin-id></code>	Inspect plugin's metadata, inputs, capabilities
<code>od plugin install <plugin-id></code>	Install from registry (or local folder / URL)
<code>od plugin apply <plugin-id> --input <key>=<value></code>	Apply a plugin with inputs
<code>od plugin upgrade <plugin-id></code>	Upgrade an installed plugin


```
od plugin uninstall <plugin-id>
```

Uninstall a plugin

Note: Every command supports `--json` for piping through `jq`/xargs into automation. **Note:** On macOS with Homebrew `cask`, `od` may resolve to Apple's built-in `octal-dump` utility — use full path or the MCP server config instead if this happens.

Development/Contribution Commands

```
gh issue list --label "good first issue" --state open --limit 20
gh issue develop <number>          # create branch and worktree
pnpm guard && pnpm typecheck        # run checks before commit
pnpm --filter @open-design/<package> test  # run package tests
```

Core Workflow (7 Stages)

```
brief → plugin → direction → design system → artifact → handoff → memory
```

Desktop App Sections

```
Home          → Overview entry point (pick skill + design system, type brief)
Automation    → Orchestrate repetitive design workflows
Design System → Distill DESIGN.md into brand contract
Plugin        → Browse/install/distribute workflow plugins
Studio        → Per-project: prototypes, HyperFrames, decks, images
```

Scale (v0.15.0)

```
259+ Skills
142+ Design Systems (brand-grade)
9-section DESIGN.md schema (portable Markdown, not theme JSON)
371+ contributors
```

Supported Coding Agents (20+ CLIs via BYOK)

```
Claude Code, Codex, Cursor, Gemini CLI, OpenCode
Qwen, Copilot, OpenClaw, Antigravity, Hermes, Kimi
... and 10+ more
```

Output Formats

Web prototypes (HTML)
Desktop/mobile prototypes
Slide decks (with slides.json sidecar for PPTX export)
Images
Videos (MP4)
HyperFrames
Export: HTML / PDF / PPTX / MP4

Plugin Structure (Minimum Requirements)

```
my-plugin/  
├─ SKILL.md           # required - executable agent contract  
└─ open-design.json   # optional - OD marketplace metadata (sidecar)
```

MCP Server Setup (Alternative to CLI)

Available in Settings → MCP server in desktop app
Provides absolute-path client-specific config snippet
Use when 'od' command conflicts with system utilities

MASTER COMPARISON — ALL 10 REPOS

Repo	Stars	Type	Primary Focus	Commands	Agents	Skills
ECC	231K	Framework	Agent optimization	94+	67	278+
Superpowers	6.3K	Methodology	Dev workflow	5+	5-10	20+
GSD Core	6.7K	Orchestrator	Context management	5+	5	Integrated
Ponytail	85K	Principle/Mode	Code minimization	10+	Custom	12
Graphify	90K	Analyzer	Knowledge graph (code)	8+	1	1

claude-obsidian	9.4K	Second brain	Personal knowledge mgmt	11+	3	15
obsidian-skills	42K	Skill pack	Obsidian CLI/format literacy	N/A	N/A	Multiple
andrej-karpathy-skills	192K	Guideline	Coding discipline	N/A	N/A	1
ui-ux-pro-max-skill	107K	Database	UI/UX design intelligence	6+	N/A	7 categories
open-design	79K	Design engine	Visual artifact generation	7+	N/A	259+

THE COMPLETE INTEGRATED STACK

7 Functional Layers

1. KNOWLEDGE LAYER

claude-obsidian + obsidian-skills

→ Builds and queries your persistent second brain

2. BEHAVIOR/DISCIPLINE LAYER

andrej-karpathy-skills

→ Fixes silent assumptions, over-engineering, goal-driven execution

3. CODE MINIMIZATION LAYER

Ponytail

→ Forces minimal, pragmatic, YAGNI-driven solutions

4. WORKFLOW LAYER

Superpowers

→ Structures brainstorm → plan → execute

5. CONTEXT MANAGEMENT LAYER

GSD Core

→ Manages long-running features across phases (Discuss→Plan→Execute→Test→Ship)

6. OPTIMIZATION & QUALITY LAYER

ECC

→ Agent performance, learned instincts, security, verification loops

7. MAPPING & DESIGN LAYER

Graphify (code structure) + ui-ux-pro-max-skill (design intelligence) + open-de:

→ Understands code structure AND generates visual/design artifacts

Full Daily Workflow (All 10 Combined)

MORNING - Context & Knowledge

1. /wiki (claude-obsidian: check vault status)
 2. ingest [new source] (claude-obsidian: absorb new info)
 3. /graphify . (map current codebase state)
 4. /ponytail ultra (set minimal-code mode)
- [Karpathy guidelines auto-applied via CLAUDE.md/plugin throughout]

PLANNING

5. /brainstorm (Superpowers: explore ideas)
6. /gsd-onboard (GSD: understand codebase)
7. /gsd-plan-feature (GSD: plan this sprint)

DESIGN (if UI/UX work involved)

8. uipro search "<design query>" (ui-ux-pro-max: find design pattern)
9. od plugin apply <plugin> (open-design: generate artifact)

EXECUTION

10. /execute-plan (Superpowers: implement)
11. /ecc:code-review (ECC: verify quality)
12. /ecc:quality-gate (ECC: final checks)

COMPLETION

13. /gsd-ready-to-ship (GSD: ship it)
14. /graphify diff (see what changed)
15. /ecc:learn (extract patterns for next time)
16. /save (claude-obsidian: file the session)

COMPLETE INSTALLATION CHECKLIST — ALL 10 REPOS

- **ECC:** `/plugin marketplace add https://github.com/affaan-m/ECC → /plugin install ecc@ecc`
- **Superpowers:** `/plugin marketplace add obra/superpowers-marketplace → /plugin install superpowers@superpowers-marketplace`
- **GSD Core:** `npx @opengsd/gsd-core@latest --install`
- **Ponytail:** `/plugin marketplace add https://github.com/DietrichGebert/ponytail → /plugin install ponytail@ponytail`
- **Graphify:** `uv tool install graphify && graphify install`
- **claude-obsidian:** `git clone https://github.com/AgriciDaniel/claude-obsidian && bash bin/setup-vault.sh`
- **obsidian-skills:** `/plugin marketplace add kepano/obsidian-skills → /plugin install obsidian@obsidian-skills`
- **andrej-karpathy-skills:** `/plugin marketplace add multica-ai/andrej-karpathy-skills`
- **ui-ux-pro-max-skill:** `/plugin marketplace add nextlevelbuilder/ui-ux-pro-max-skill → /plugin install ui-ux-pro-max@ui-ux-pro-max-skill`
- **open-design:** `git clone https://github.com/nexu-io/open-design.git && pnpm install`
- Set environment variables for all
- Test each with its respective health/status command
- Bookmark this file for daily reference

GRAND TOTAL COMMAND COUNT — ALL 10 REPOS

Repo	Commands	Agents	Skills	Subtotal
ECC	94+	67	278+	439+
Superpowers	5+	5-10	20+	30+
GSD Core	5+	5	Integrated	10+
Ponytail	10+	Custom	12	22+

Graphify	8+	1	1	10+
claude-obsidian	11+	3	15	29+
obsidian-skills	N/A	N/A	Multiple	~5+
andrej-karpathy-skills	N/A	N/A	1	1+
ui-ux-pro-max-skill	6+	N/A	7 categories	13+
open-design	7+	N/A	259+	266+
GRAND TOTAL	146+	88-93	595+	~825+

All 10 repos combined provide 825+ commands, agents, and skills. This single file is your complete, definitive AI coding assistant command reference — covering optimization, workflow, context, minimalism, code mapping, personal knowledge management, coding discipline, and design generation.

Bookmark and master these tools for peak productivity! 🚀